

B.TECH · FIRST YEAR · PROGRAMMING

Arrays & Strings in C

Programming for Problem Solving



Unit III – Complete Study Module

A fully structured, exam-oriented reference covering theory, diagrams, programs, questions, and quick revision material.

1D Arrays

2D Arrays

Matrix Operations

Strings in C

String Functions

Memory Models

Q & A

Quick Revision

Prepared for B.Tech 1st Year Students | C Language Series | Exam-Ready Edition



Table of Contents

Section	Topic	Page
1	Introduction to Arrays	3
2	Array Declaration and Initialization	4
3	Array Indexing	5
4	Memory Representation of Arrays	6
5	Programs Using 1D Arrays	7–8
6	Introduction to 2D Arrays	9
7	Matrix Representation	10
8	Programs Using 2D Arrays	11–12
9	Introduction to Strings	13
10	String Representation in C	14
11	String Operations	15
12	String Handling Programs	16–17
13	Questions & Answers	18–20
14	Quick Revision Sheet	21



How to Use This Module: Each section follows the same structure – Definition → Theory → Syntax → Diagram → Programs. Use the Quick Revision Sheet (Section 14) before exams and the Q&A Section (13) for answer writing practice.

1.1 Definition

An **Array** is a collection of a **fixed number of elements of the same data type**, stored in **contiguous (adjacent) memory locations** and accessed using a single variable name with an index (subscript).

1.2 Why Do We Need Arrays?

Consider storing marks of 100 students. Without arrays, you would need 100 separate variables: `m1`, `m2`, `m3`, ..., `m100`. With arrays, a single declaration handles all: `int marks[100];`.

Without Arrays

`int m1, m2, m3, ..., m100;` – Tedious, error-prone, and not scalable.

With Arrays

`int marks[100];` – Single declaration, easy traversal using loops.

1.3 Key Characteristics

- **Homogeneous:** All elements must be of the same data type.
- **Contiguous Memory:** Elements are stored in adjacent memory cells.
- **Fixed Size:** The size must be declared at compile time (in standard C).
- **Zero-Based Indexing:** The first element is at index 0.
- **Random Access:** Any element can be accessed directly in $O(1)$ time using its index.

1.4 Types of Arrays

Type	Description	Example
One-Dimensional (1D)	Linear list of elements	<code>int arr[5];</code>
Two-Dimensional (2D)	Table/Matrix of elements	<code>int mat[3][3];</code>
Multi-Dimensional	Arrays with 3 or more dimensions	<code>int cube[2][3][4];</code>

1.5 Advantages and Limitations

✓ Advantages

- Simple and easy to use
- Efficient memory usage (contiguous)

✗ Limitations

- Fixed size – cannot grow dynamically
- Insertion/deletion is expensive $O(n)$

- Fast random access $O(1)$
- Easy to traverse with loops
- Supports sorting & searching

- Memory wastage if array not fully used
- All elements must be the same type

2

Array Declaration and Initialization

2.1 Declaration Syntax

```
/* General Syntax */  
data_type  array_name [ size ];  
  
/* Examples */  
int        marks[5];           // 5 integers  
float      salary[100];        // 100 floating-point numbers  
char       name[20];           // 20 characters (used for strings)  
double     temp[7];            // 7 double values
```

2.2 Initialization Methods

Method 1 – Initialize at Declaration

```
int arr[5] = {10, 20, 30, 40, 50};  
// arr[0]=10, arr[1]=20, arr[2]=30, arr[3]=40, arr[4]=50
```

Method 2 – Partial Initialization (Remaining become 0)

```
int arr[5] = {10, 20};  
// arr[0]=10, arr[1]=20, arr[2]=0, arr[3]=0, arr[4]=0
```

Method 3 – Without Size (Compiler determines size)

```
int arr[] = {5, 10, 15, 20};  
// Compiler sets size = 4 automatically
```

Method 4 – Initialize All to Zero

```
int arr[5] = {0};  
// All 5 elements set to 0
```

Method 5 – Run-time Input

```
int arr[5], i;
for(i = 0; i < 5; i++) {
    printf("Enter element %d: ", i+1);
    scanf("%d", &arr[i]);
}
```

2.3 Rules for Array Declaration

- Array size must be a **positive integer constant** (in standard C89).
- Array name follows the same rules as variable names.
- The size cannot be zero or negative.
- In C99 and later, Variable Length Arrays (VLAs) are allowed.

2.4 Comparison Table

Method	Size Known?	Values Set At	Remaining Elements
Full Init	Yes/No	Compile time	All defined
Partial Init	Yes	Compile time	Set to 0
Without Size	Auto	Compile time	None
Run-time Input	Yes	Run time	Garbage if not set

3

Array Indexing

3.1 Definition

Array Indexing refers to accessing individual elements of an array using a numeric subscript called an **index**. In C, arrays use **zero-based indexing** — the first element is at index 0.

3.2 Indexing Syntax

```
/* Access an element */  
array_name[index]  
  
int arr[5] = {10, 20, 30, 40, 50};  
printf("%d", arr[0]);    // Output: 10  (first element)  
printf("%d", arr[4]);    // Output: 50  (last element)  
arr[2] = 99;             // Modify third element → arr = {10, 20, 99, 40, 50}
```

3.3 Indexing Diagram

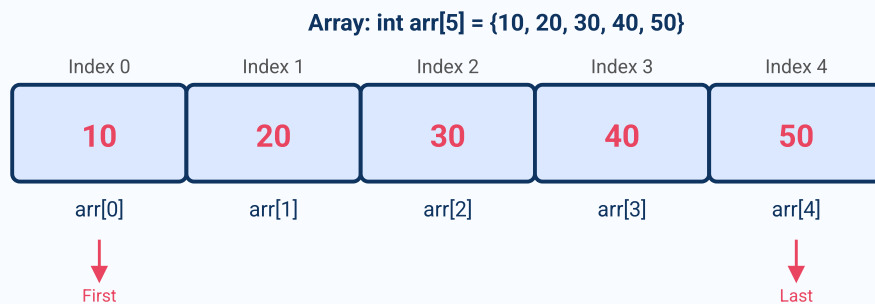


Fig 3.1 – Zero-Based Array Indexing in C

3.4 Index Calculation Formula

Address of arr[i] = Base Address + $i \times \text{sizeof}(\text{data_type})$

Example: If arr starts at address 1000 and is of type int (4 bytes):

Address of arr[3] = $1000 + 3 \times 4 = 1012$

3.5 Valid vs Invalid Indexing

Expression	Valid?	Reason
arr[0]	✔ Yes	First element, always valid
arr[4]	✔ Yes	Last element (size 5)
arr[-1]	✘ No	Negative index – undefined behavior
arr[5]	✘ No	Out of bounds for size 5 array
arr[i] ($0 \leq i < 5$)	✔ Yes	Valid loop-based access

4

Memory Representation of Arrays

4.1 How Arrays Are Stored in Memory

When an array is declared, the compiler allocates a **contiguous block of memory**. Each element occupies a fixed number of bytes depending on its data type. The name of the array acts as a **pointer to the base address** (address of first element).

4.2 Memory Size Calculation

Total Memory = Number of Elements × Size of each element (bytes)

Example: `int arr[5];` → $5 \times 4 = 20$ bytes

Example: `float arr[10];` → $10 \times 4 = 40$ bytes

4.3 Memory Layout Diagram

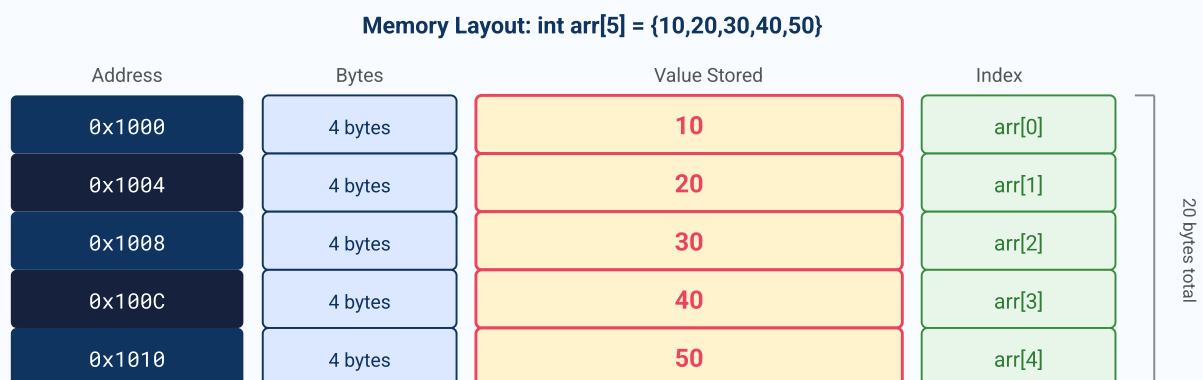


Fig 4.1 – Contiguous Memory Layout of `int arr[5]` (4 bytes each)

4.4 Key Memory Concepts

- **Base Address:** Address of `arr[0]` – the starting point of the array.
- **Array Name as Pointer:** `arr` is equivalent to `&arr[0]`.
- **Element Address Formula:** $\text{Address}(\text{arr}[i]) = \text{BaseAddr} + i \times \text{sizeof}(\text{type})$
- **sizeof() operator:** `sizeof(arr)` gives total bytes; `sizeof(arr)/sizeof(arr[0])` gives number of elements.

```
/* Finding size of array using sizeof */  
int arr[5] = {10, 20, 30, 40, 50};  
int n = sizeof(arr) / sizeof(arr[0]);
```

```
printf("Number of elements: %d\n", n); // Output: 5  
printf("Total bytes: %d\n", sizeof(arr)); // Output: 20
```

5 Programs Using 1D Arrays

Program 5.1 – Input and Display Array Elements

```
/* Program to read and print array elements */
#include <stdio.h>

int main() {
    int arr[5], i;

    /* Input */
    printf("Enter 5 integers:\n");
    for(i = 0; i < 5; i++) {
        scanf("%d", &arr[i]);
    }

    /* Output */
    printf("Array elements: ");
    for(i = 0; i < 5; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    return 0;
}
```

Program 5.2 – Sum and Average of Array Elements

```
#include <stdio.h>

int main() {
    int arr[5], i, sum = 0;
    float avg;

    printf("Enter 5 numbers: ");
    for(i = 0; i < 5; i++) scanf("%d", &arr[i]);

    for(i = 0; i < 5; i++) sum += arr[i];
    avg = (float)sum / 5;

    printf("Sum      = %d\n", sum);
    printf("Average = %.2f\n", avg);
}
```

```
    return 0;
}
```

Program 5.3 – Find Maximum and Minimum

```
#include <stdio.h>

int main() {
    int arr[5] = {34, 12, 78, 5, 56};
    int i, max = arr[0], min = arr[0];

    for(i = 1; i < 5; i++) {
        if(arr[i] > max) max = arr[i];
        if(arr[i] < min) min = arr[i];
    }
    printf("Maximum = %d\n", max); // Output: 78
    printf("Minimum = %d\n", min); // Output: 5
    return 0;
}
```

Program 5.4 – Reverse an Array

```
#include <stdio.h>

int main() {
    int arr[5] = {1, 2, 3, 4, 5};
    int i, temp, start = 0, end = 4;

    while(start < end) {
        temp = arr[start];
        arr[start] = arr[end];
        arr[end] = temp;
        start++; end--;
    }

    printf("Reversed: ");
    for(i = 0; i < 5; i++) printf("%d ", arr[i]);
    // Output: 5 4 3 2 1
    return 0;
}
```

Program 5.5 – Linear Search

```
#include <stdio.h>

int main() {
```

```
int arr[5] = {10, 25, 35, 45, 55};  
int key = 35, found = -1, i;  
  
for(i = 0; i < 5; i++) {  
    if(arr[i] == key) { found = i; break; }  
}  
  
if(found != -1)  
    printf("%d found at index %d\n", key, found);  
else  
    printf("%d not found\n", key);  
return 0;  
}
```

6 Introduction to 2D Arrays

6.1 Definition

A **Two-Dimensional (2D) Array** is an array of arrays — a collection of elements arranged in **rows and columns** (like a matrix or table). It requires **two indices**: one for the row and one for the column.

6.2 Declaration Syntax

```
/* General Syntax */
data_type array_name [rows][columns];

/* Examples */
int mat[3][3];           // 3x3 matrix of integers (9 elements)
float grid[4][5];        // 4 rows, 5 columns (20 elements)
char board[8][8];        // 8x8 board (e.g., chessboard)
```

6.3 Initialization

```
/* Method 1: Row-wise initialization */
int mat[2][3] = {{1,2,3}, {4,5,6}};

/* Method 2: Flat initialization */
int mat[2][3] = {1,2,3,4,5,6};

/* Accessing element at row r, column c */
printf("%d", mat[1][2]); // Prints 6 (row 1, col 2)
```

6.4 2D Array Traversal Diagram

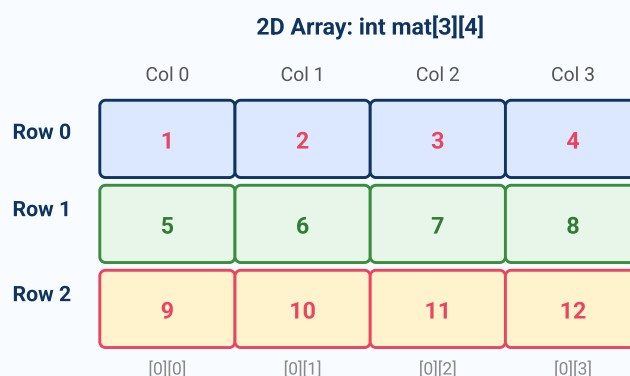


Fig 6.1 – 2D Array (3 rows × 4 cols = 12 elements)

6.5 Key Properties

Property	1D Array	2D Array
Dimensions	1	2
Indices needed	1 (row only)	2 (row & column)
Declaration	<code>int a[n];</code>	<code>int a[r][c];</code>
Total Elements	n	r × c
Memory (int)	n × 4 bytes	r × c × 4 bytes
Primary Use	Lists, sequences	Matrices, tables, images

7.1 Row-Major Order (C Language Default)

In C, 2D arrays are stored in **Row-Major Order** — elements of each row are stored consecutively in memory. Row 0 is stored first, then Row 1, then Row 2, and so on.

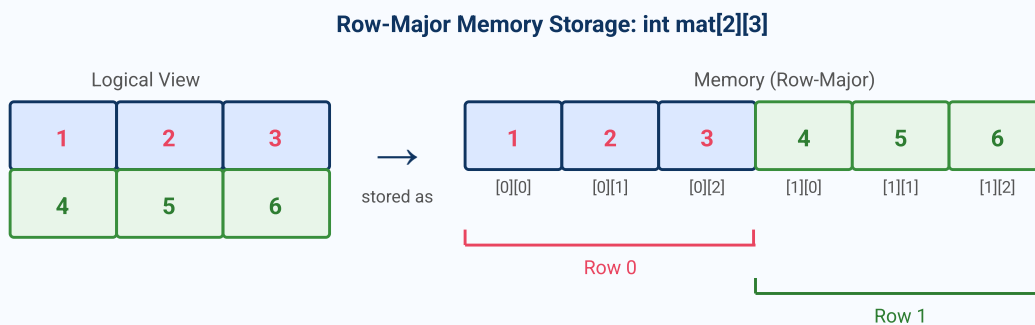


Fig 7.1 – Row-Major Order: C stores entire Row 0, then Row 1

7.2 Address Formula (2D Array)

Address of $\text{mat}[i][j]$ = $\text{BaseAddr} + (i \times \text{COLS} + j) \times \text{sizeof}(\text{data_type})$

Example: $\text{mat}[3][4]$, BaseAddr = 2000, int (4 bytes)

Address of $\text{mat}[1][2]$ = $2000 + (1 \times 4 + 2) \times 4 = 2000 + 24 = \mathbf{2024}$

7.3 Column-Major Order (Concept — Used in FORTRAN)

In **Column-Major Order**, elements are stored column by column. C does *not* use this — it is mentioned here for comparison.

Storage order for $\text{mat}[2][3]$: $[0][0]$, $[1][0]$, $[0][1]$, $[1][1]$, $[0][2]$, $[1][2]$

Order	Storage Sequence	Used In
Row-Major	Row 0 complete → Row 1 → ...	C, C++, Java
Column-Major	Col 0 complete → Col 1 → ...	FORTRAN, MATLAB

Program 8.1 – Input and Display a Matrix

```
#include <stdio.h>

int main() {
    int mat[3][3], i, j;

    printf("Enter 9 elements (3x3 matrix):\n");
    for(i = 0; i < 3; i++)
        for(j = 0; j < 3; j++)
            scanf("%d", &mat[i][j]);

    printf("Matrix:\n");
    for(i = 0; i < 3; i++) {
        for(j = 0; j < 3; j++)
            printf("%4d", mat[i][j]);
        printf("\n");
    }
    return 0;
}
```

Program 8.2 – Matrix Addition

```
#include <stdio.h>

int main() {
    int A[2][2] = {{1,2},{3,4}};
    int B[2][2] = {{5,6},{7,8}};
    int C[2][2], i, j;

    for(i = 0; i < 2; i++)
        for(j = 0; j < 2; j++)
            C[i][j] = A[i][j] + B[i][j];

    printf("Sum Matrix:\n");
    for(i = 0; i < 2; i++) {
        for(j = 0; j < 2; j++) printf("%4d", C[i][j]);
        printf("\n");
    }
    /* Output: 6 8
               10 12 */
}
```

```
    return 0;
}
```

Program 8.3 – Matrix Multiplication

```
#include <stdio.h>

int main() {
    int A[2][2] = {{1,2},{3,4}};
    int B[2][2] = {{5,6},{7,8}};
    int C[2][2] = {{0,0},{0,0}};
    int i, j, k;

    for(i = 0; i < 2; i++)
        for(j = 0; j < 2; j++)
            for(k = 0; k < 2; k++)
                C[i][j] += A[i][k] * B[k][j];

    printf("Product:\n");
    for(i = 0; i < 2; i++) {
        for(j = 0; j < 2; j++) printf("%5d", C[i][j]);
        printf("\n");
    }
    /* Output: 19 22
               43 50 */
    return 0;
}
```

Program 8.4 – Transpose of a Matrix

```
#include <stdio.h>

int main() {
    int mat[3][3] = {{1,2,3},{4,5,6},{7,8,9}};
    int i, j;

    printf("Transpose:\n");
    for(i = 0; i < 3; i++) {
        for(j = 0; j < 3; j++)
            printf("%4d", mat[j][i]);    // Note: j,i swapped
        printf("\n");
    }
    /* Output:  1  4  7
               2  5  8
               3  6  9 */
}
```

```
return 0;
```

```
}
```

9.1 Definition

A **String** in C is a **sequence of characters terminated by a null character '\0'**. C does not have a built-in string type; strings are implemented as **character arrays**.

9.2 Why Strings Are Important

Strings are used everywhere in programs — storing names, processing text, reading user input, file handling, and so on. Understanding strings in C gives deep insight into how text data is managed at the memory level.

9.3 Declaration and Initialization

```
/* Method 1: Character array with explicit size */
char name[10] = {'H','e','l','l','o','\0'};

/* Method 2: Using string literal (recommended) */
char name[10] = "Hello";

/* Method 3: Without size (auto-calculated) */
char name[] = "Hello"; // Size = 6 (5 chars + '\0')

/* Reading a string from user */
char str[50];
scanf("%s", str); // Reads until whitespace
gets(str); // Reads entire line (use carefully)
```

9.4 Array vs String Comparison

Feature	Integer Array	String (char array)
Data Type	int / float	char
Terminator	None needed	Must end with '\0'
Format Specifier	%d, %f	%s or %c
Library Functions	Custom only	string.h functions
Direct I/O	Element-wise	Whole string with %s

Feature	Integer Array	String (char array)
Comparison	== operator	strcmp() function

10 String Representation in C

10.1 The Null Terminator '\0'

Every C string ends with a special character called the **null terminator** ('\0') — ASCII value 0. This signals the end of the string to all string-handling functions. Without '\0', string functions would read beyond the intended end of the string, causing undefined behavior.

10.2 String Memory Diagram

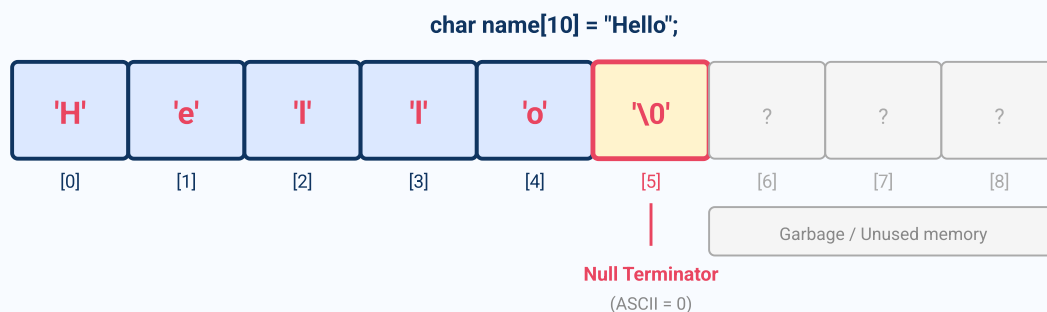


Fig 10.1 – String Storage in Memory (char array size 9)

10.3 String Traversal Diagram

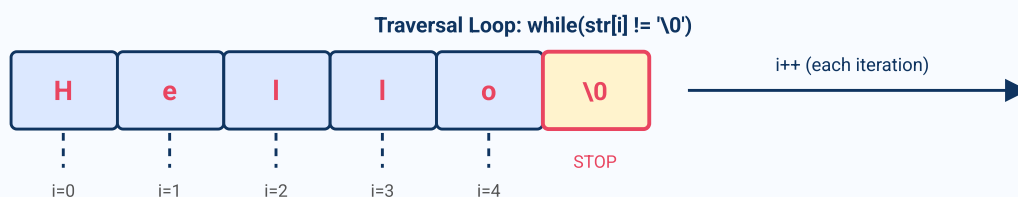


Fig 10.2 – String Traversal using Index

```
/* String traversal using index */  
char str[] = "Hello";  
int i = 0;  
while(str[i] != '\0') {  
    printf("%c", str[i]);  
    i++;  
}
```

```
/* Equivalent using pointer */
```

```
char *p = str;
```

```
while(*p != '\0') { printf("%c", *p); p++; }
```

11.1 Standard Library – <string.h>

The C standard library provides a set of functions for string manipulation, all declared in <string.h>.

Function	Prototype	Description	Example
strlen()	int strlen(char *s)	Returns length (excluding '\0')	strlen("Hello") → 5
strcpy()	char* strcpy(char *d, char *s)	Copies s into d	strcpy(dest, "Hi")
strcat()	char* strcat(char *d, char *s)	Appends s to end of d	strcat("Hi", " World")
strcmp()	int strcmp(char *s1, char *s2)	Compares two strings lexicographically	strcmp("abc","abc") → 0
strrev()	char* strrev(char *s)	Reverses a string in place	strrev("Hello") → "olleH"
strupr()	char* strupr(char *s)	Converts to uppercase	strupr("hello") → "HELLO"
strlwr()	char* strlwr(char *s)	Converts to lowercase	strlwr("HELLO") → "hello"
strchr()	char* strchr(char *s, int c)	Finds first occurrence of c in s	strchr("hello",'l') → "llo"
strstr()	char* strstr(char *s1, char *s2)	Finds s2 in s1	strstr("hello","ell")

11.2 strcmp() Return Values

Condition	Return Value	Meaning
s1 == s2	0	Both strings are identical
s1 < s2	Negative (<0)	s1 comes before s2 alphabetically
s1 > s2	Positive (>0)	s1 comes after s2 alphabetically

11.3 Manual Implementation – strlen()

```
/* Custom strlen function */
int myStrlen(char str[]) {
    int count = 0;
    while(str[count] != '\0') count++;
    return count;
}
```



```
}
```

```
int main() {  
    char s[] = "Programming";  
    printf("Length = %d\n", myStrlen(s)); // Output: 11  
    return 0;  
}
```

12 String Handling Programs

Program 12.1 – Using Built-in String Functions

```
#include <stdio.h>
#include <string.h>

int main() {
    char s1[20] = "Hello";
    char s2[20] = "World";
    char s3[40];

    printf("Length of s1: %d\n", strlen(s1));    // 5
    strcpy(s3, s1);
    printf("Copied s3: %s\n", s3);              // Hello
    strcat(s3, s2);
    printf("Concatenated: %s\n", s3);           // HelloWorld
    printf("Compare: %d\n", strcmp(s1,s2));    // negative
    return 0;
}
```

Program 12.2 – Check if String is Palindrome

```
#include <stdio.h>
#include <string.h>

int main() {
    char str[50], rev[50];
    printf("Enter a string: ");
    scanf("%s", str);

    strcpy(rev, str);
    strrev(rev);

    if(strcmp(str, rev) == 0)
        printf("%s is a palindrome\n", str);
    else
        printf("%s is NOT a palindrome\n", str);
    return 0;
}

/* Input: madam → Output: madam is a palindrome */
```

Program 12.3 – Count Vowels and Consonants

```

#include <stdio.h>
#include <ctype.h>

int main() {
    char str[] = "Hello World";
    int v = 0, c = 0, i = 0;

    while(str[i] != '\0') {
        char ch = tolower(str[i]);
        if(ch=='a' || ch=='e' || ch=='i' || ch=='o' || ch=='u') v++;
        else if(ch >= 'a' && ch <= 'z') c++;
        i++;
    }
    printf("Vowels: %d, Consonants: %d\n", v, c);
    // Output: Vowels: 3, Consonants: 7
    return 0;
}

```

Program 12.4 – Reverse a String Without Library Function

```

#include <stdio.h>
#include <string.h>

void reverseStr(char str[]) {
    int n = strlen(str);
    int start = 0, end = n - 1;
    char temp;
    while(start < end) {
        temp = str[start]; str[start] = str[end]; str[end] = temp;
        start++; end--;
    }
}

int main() {
    char s[] = "Programming";
    reverseStr(s);
    printf("Reversed: %s\n", s); // gnimmargorP
    return 0;
}

```

13 Questions & Answers

Q1. What is an array in C? Give an example.

2 Marks

Answer:

An array in C is a collection of a fixed number of elements of the same data type stored in contiguous memory locations. All elements are accessed using a single variable name with an index (subscript). Arrays follow zero-based indexing – the first element is at index 0.

```
int marks[5] = {85, 90, 75, 60, 95};  
/* marks[0]=85, marks[1]=90, ..., marks[4]=95 */
```

Q2. What is a string in C? How is it different from a character array?

2 Marks

Answer:

A string in C is a sequence of characters terminated by a null character '\0'. It is stored as a character array where the last character is always '\0'. A character array becomes a string only when it contains '\0' at the end. Without '\0', it is just a plain array of characters.

```
char arr[] = {'H', 'i'};           // character array (NOT a string)  
char str[] = {'H', 'i', '\0'};    // string  
char str2[] = "Hi";               // string (auto '\0' added)
```

Q3. What is the difference between a 1D array and a 2D array?

2 Marks

Answer:

A 1D array is a linear list of elements accessed with one index (e.g., `arr[i]`). It is used for simple lists. A 2D array is a table of elements (rows and columns) accessed with two indices (e.g., `mat[i][j]`). It is used for matrices, grids, and tabular data. The total memory for a 1D array of size n is $n \times \text{sizeof}(\text{type})$, while for a 2D array of $r \times c$ it is $r \times c \times \text{sizeof}(\text{type})$.

```
int a[5];           // 1D: 5 elements
int b[3][4];        // 2D: 3 rows × 4 cols = 12 elements
```

Q4. State two advantages and two limitations of arrays.

2 Marks

Answer:

Advantages: (1) Arrays provide fast random access to any element in $O(1)$ time using its index. (2) They use contiguous memory, making them cache-friendly and easy to iterate.

Limitations: (1) Arrays have a fixed size that cannot be changed at runtime. (2) Insertion and deletion of elements in the middle is expensive — $O(n)$ — as elements must be shifted.

Q5. Explain array indexing with diagram and example program.

5 Marks

Answer:

Array indexing is the process of accessing individual elements of an array using a numeric subscript. In C, **zero-based indexing** is used — the index of the first element is 0, the second is 1, and so on up to (size-1).

Syntax: array_name[index]

Memory: The address of element arr[i] = Base Address + i × sizeof(data_type). For example, if arr starts at 2000 and is of type int (4 bytes), then arr[3] is at address 2000 + 3×4 = 2012.

Important Rule: Valid indices are 0 to (n-1). Accessing arr[-1] or arr[n] causes undefined behavior in C. The compiler does NOT perform bounds checking automatically.

```
#include <stdio.h>

int main() {
    int arr[5] = {10, 20, 30, 40, 50};
    int i;

    /* Access by index */
    printf("First element : arr[0] = %d\n", arr[0]); // 10
    printf("Third element : arr[2] = %d\n", arr[2]); // 30
    printf("Last element  : arr[4] = %d\n", arr[4]); // 50

    /* Modify using index */
    arr[2] = 99;

    /* Print all using loop */
    printf("\nAll elements:\n");
    for(i = 0; i < 5; i++)
        printf("arr[%d] = %d\n", i, arr[i]);

    return 0;
}
```

Output: arr[0]=10, arr[1]=20, arr[2]=99, arr[3]=40, arr[4]=50

Q6. Write a program to perform matrix multiplication using 2D arrays.

5 Marks

Answer:

Matrix multiplication of matrices A (m×n) and B (n×p) results in matrix C (m×p) where each element C[i][j] is the dot product of row i of A and column j of B. The formula is: $C[i][j] = \sum A[i][k] \times B[k][j]$ for k from 0 to n-1.

Condition: Number of columns of A must equal number of rows of B.

```
#include <stdio.h>

#define R1 2
#define C1 3
#define C2 2

int main() {
    int A[R1][C1] = {{1,2,3},{4,5,6}};
    int B[C1][C2] = {{7,8},{9,10},{11,12}};
    int C[R1][C2] = {0};
    int i, j, k;

    for(i = 0; i < R1; i++)
        for(j = 0; j < C2; j++)
            for(k = 0; k < C1; k++)
                C[i][j] += A[i][k] * B[k][j];

    printf("Product Matrix:\n");
    for(i = 0; i < R1; i++) {
        for(j = 0; j < C2; j++) printf("%6d", C[i][j]);
        printf("\n");
    }
    return 0;
}

/* Output:  58  64
           139 154 */
```

Q7. Explain strings in C with memory representation. Write a program demonstrating string operations.

5 Marks

Answer:

A string in C is a one-dimensional character array terminated by the null character '\0' (ASCII 0). Strings must be declared with enough space to hold all characters plus the null terminator. The C compiler automatically appends '\0' when a string literal is used.

Memory: For `char s[10] = "Hello"`, memory stores: H(72), e(101), l(108), l(108), o(111), \0(0), and 4 unused bytes. Each character occupies 1 byte.

Important string functions: `strlen()` – length, `strcpy()` – copy, `strcat()` – concatenate, `strcmp()` – compare.

```
#include <stdio.h>
#include <string.h>

int main() {
    char s1[30] = "C Language";
    char s2[30] = " Programming";
    char s3[60];

    printf("Original   : %s\n", s1);
    printf("Length     : %d\n", strlen(s1));           // 10
    strcpy(s3, s1);
    printf("Copied      : %s\n", s3);                 // C Language
    strcat(s3, s2);
    printf("Concat     : %s\n", s3);                 // C Language Programming
    printf("Compare    : %d\n", strcmp(s1,s2));       // positive (C>space)
    strupr(s1);
    printf("Uppercase  : %s\n", s1);                 // C LANGUAGE
    return 0;
}
```

Q8. Write a program to sort an array in ascending order using Bubble Sort.

5 Marks

Answer:

Bubble Sort repeatedly compares adjacent elements and swaps them if they are in the wrong order. After each full pass, the largest element "bubbles up" to the end. For n elements, $n-1$ passes are required in the worst case.

Time Complexity: $O(n^2)$ worst and average case. **Space Complexity:** $O(1)$ – in-place sorting.


```
#include <stdio.h>

int main() {
    int arr[] = {64, 34, 25, 12, 22, 11, 90};
    int n = 7, i, j, temp;

    /* Bubble Sort */
    for(i = 0; i < n-1; i++) {
        for(j = 0; j < n-1-i; j++) {
            if(arr[j] > arr[j+1]) {
                temp = arr[j];
                arr[j] = arr[j+1];
                arr[j+1] = temp;
            }
        }
    }

    printf("Sorted array: ");
    for(i = 0; i < n; i++) printf("%d ", arr[i]);
    // Output: 11 12 22 25 34 64 90
    return 0;
}
```

📌 Array Syntax

- `int arr[n];` – declaration
- `int arr[5]={1,2,3,4,5};` – init
- `arr[i]` – access element `i`
- Index range: 0 to `n-1`
- `sizeof(arr)/sizeof(arr[0])` → `n`

📌 2D Array Syntax

- `int mat[r][c];`
- `mat[i][j]` – access
- Total elements = $r \times c$
- Stored in row-major order in C
- Address = Base + $(i \times c + j) \times \text{size}$

📌 String Basics

- `char s[n]="text";`
- Always ends with `'\0'`
- Input: `scanf("%s", s);`
- Output: `printf("%s", s);`
- Include `<string.h>`

📌 Key String Functions

- `strlen(s)` → length
- `strcpy(d, s)` → copy
- `strcat(d, s)` → append
- `strcmp(s1, s2)` → compare
- `strrev(s)` → reverse

📌 Memory Points

- `int`: 4 bytes, `char`: 1 byte
- `float`: 4 bytes, `double`: 8 bytes
- Array name = base address
- C: row-major storage
- String extra byte for `'\0'`

📌 Common Programs

- Sum / Average of array
- Find Max / Min
- Reverse array
- Bubble sort
- Matrix add / multiply
- Palindrome check
- Count vowels

Important Formulae at a Glance

Concept	Formula
1D Array total bytes	$n \times \text{sizeof}(\text{type})$
Address of <code>arr[i]</code>	$\text{Base} + i \times \text{sizeof}(\text{type})$
2D Array total bytes	$\text{rows} \times \text{cols} \times \text{sizeof}(\text{type})$

Concept	Formula
Address of <code>mat[i][j]</code>	$\text{Base} + (i \times \text{cols} + j) \times \text{sizeof}(\text{type})$
String length	Number of characters before <code>'\0'</code>
char array size needed	<code>strlen + 1</code> (for <code>'\0'</code>)

Exam Tips:

- Always declare array with size one more than expected for strings.
- In 2D arrays, traverse using **nested for loops** – outer for rows, inner for columns.
- Use `sizeof(arr)/sizeof(arr[0])` to find array length without hardcoding.
- `strcmp()` returns 0 for equal strings – do not use `==` for string comparison.
- Draw memory diagrams in exams – they earn full marks even if code is partially incorrect.

Textbook References

- 1 **"The C Programming Language"** – Brian W. Kernighan & Dennis M. Ritchie, Prentice Hall (1988). [Primary textbook – Chapters 5 & 6 cover arrays and strings in depth.]
- 2 **"Schaum's Outline of Programming with C"** – Byron S. Gottfried, McGraw-Hill (1996). [Excellent problem-based coverage with solved examples.]
- 3 **"Computing Fundamentals and C Programming"** – E. Balagurusamy, McGraw-Hill (2008). [Recommended for B.Tech – student-friendly explanations.]
- 4 **"Programming in C"** – Rema Theraja, Oxford University Press (2016). [Good reference for string operations and arrays.]
- 5 **"C Programming: A Problem Solving Approach"** – Forouzan, Gilberg, Prasad, Cengage (3rd Ed.). [Problem-solving focus; aligns with B.Tech curriculum.]

— End of Unit III Study Module —

Programming for Problem Solving · B.Tech 1st Year · Arrays & Strings in C